

Secure Healthcare Data-Access Portal Platform

A multi-tenant, PHI-grade web portal platform — two Angular portals over a private, authorizer-gated API, a standards-based FHIR clinical-data explorer, claim-based RBAC, and cross-cloud delivery as code.

Angular

OIDC / PKCE

Claim-based RBAC

Private API Gateway

Lambda authorizer

HL7 FHIR R4

HITRUST-aligned

Multi-tenant

AWS SAM

CloudFront + WAF

CloudFormation IaC

Cross-cloud CI/CD

Shared design-system

Okta auth library

Expose regulated health data to many audiences — without compromising trust

A healthcare-data organization needed to serve **protected health information (PHI)** — member records and clinical data — to internal staff, providers, and third-party developers through the web. The bar for security, compliance, and standards conformance was absolute, and the same platform had to scale from one internal console to a public developer ecosystem.

Different audiences, one data estate

Internal staff needed an admin console; external partners needed a self-service portal to register apps and test APIs against a sandbox — without maintaining divergent codebases.

PHI raises the bar on everything

Every screen and API call touches regulated data. Access had to be strongly authenticated, finely authorized per user and per tenant, auditable end-to-end, and the API itself unreachable from the public internet.

Clinical data must speak a standard

Consumers expected **HL7 FHIR** resources — patients, conditions, encounters, medications, labs, procedures, care plans — not a bespoke schema.

Many environments & tenants, one artifact; governed delivery

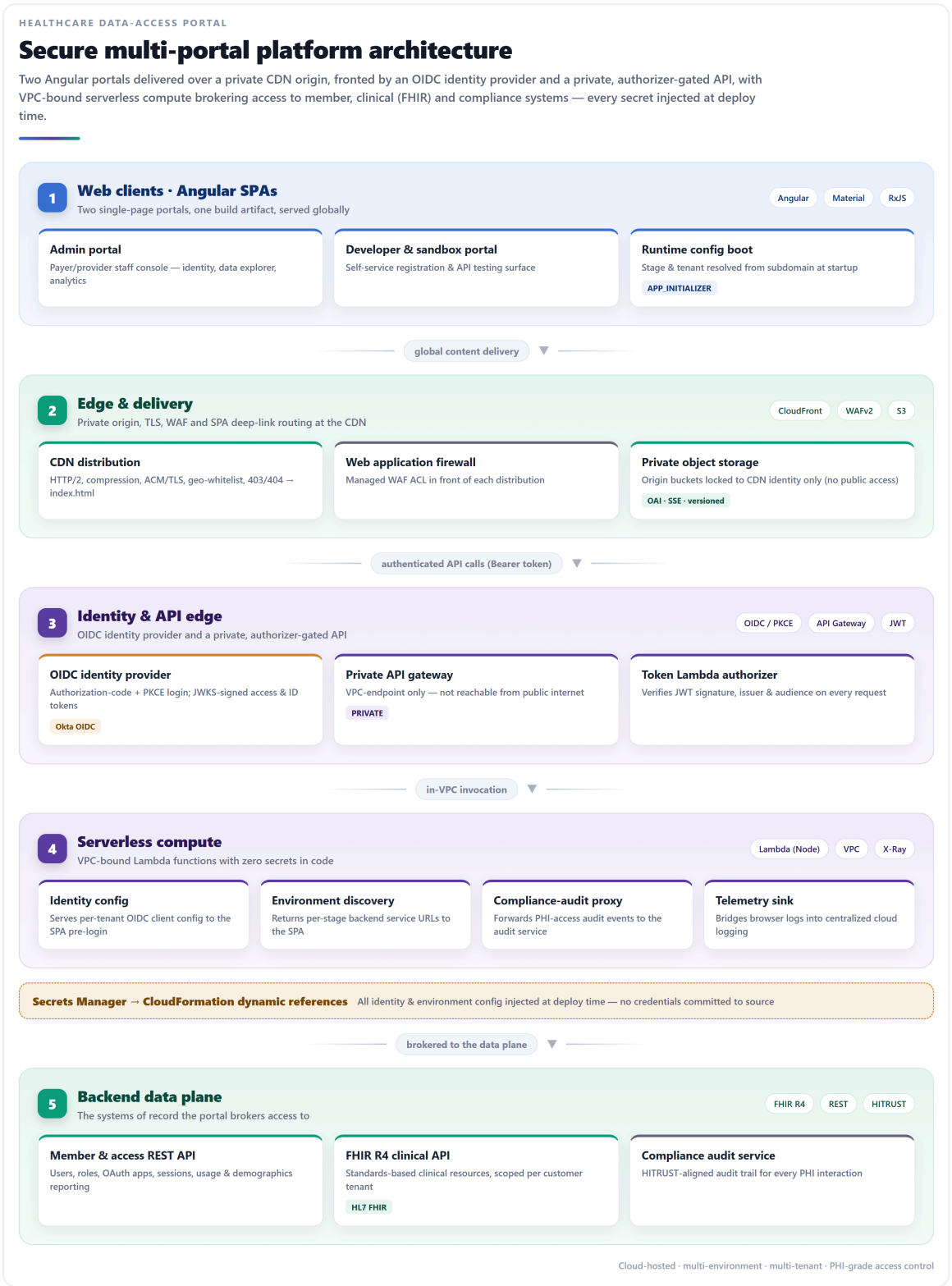
The app had to run across dev / QA / UAT / production — split across separate cloud accounts and multiple tenants — ideally from one immutable build, with infrastructure and releases codified and promoted through approval gates.

Two portals over a shared foundation — secure, standards-based, delivered as code

A two-portal platform engineered so that security, multi-tenancy, and repeatable delivery are structural rather than bolted on.

- ✓ **One build, every environment.** A single immutable Angular artifact resolves its stage, tenant, backend URLs, and identity config **at runtime from the request subdomain** — so it promotes unchanged from dev to production, and sandbox vs. production is a runtime distinction.
- ✓ **OIDC + PKCE with claim-based RBAC.** A fine-grained permission claim in the signed token drives route guards and menu rendering; the UI only shows what a user is authorized to use.
- ✓ **Private, authorizer-gated API.** The API gateway is reachable only through an in-VPC endpoint; a token Lambda authorizer independently re-verifies every JWT server-side — client checks are never trusted alone.
- ✓ **Standards-based FHIR explorer.** Patient search and profile open onto **15 HL7 FHIR R4 resource types**, paged and searchable, every query scoped to the signed-in tenant.
- ✓ **Self-service admin & analytics.** Full RBAC, OAuth client-app registration, plus API-usage and member-demographics dashboard suites on a reusable filter-and-chart pattern.
- ✓ **Delivered as code.** Reusable nested-stack IaC provisions the estate; a cross-cloud pipeline builds once and promotes through gated environments; shared Angular design-system + auth libraries give both portals one maintained foundation.

Secure multi-portal platform architecture



Authentication & runtime configuration

AUTHENTICATION & RUNTIME CONFIGURATION

One build, every environment, zero secrets shipped

A single static Angular artifact boots, discovers its environment and identity config from the subdomain, authenticates via OIDC + PKCE, then lets a signed permission claim drive route guards and menus on the client — while the private API gateway independently re-verifies every token server-side.

1 Zero-config SPA boot

One static build serves every stage — environment resolved at runtime

Load SPA from CDN

Same immutable artifact for dev / qa / uat / prod

Derive stage & tenant

Parsed from the request subdomain (incl. sandbox mode)

APP_INITIALIZER

Fetch runtime config

Environment + OIDC client config pulled from the config API

redirect to identity provider ▼

2 OIDC authentication

Authorization-code flow with PKCE — no secrets in the browser

Login (PKCE)

Redirect to OIDC provider; code exchanged for tokens

Okta OIDC

Tokens returned

Signed access + ID tokens land on the callback route

Decode claims

Tenant, roles & fine-grained permission claims extracted

userPermissions

claims drive the whole UI ▼

3 Client-side authorization

The permission claim decides what a user can see and do

CanActivate

RBAC

idle timeout

Route guards

Every feature route gated; missing permission → unauthorized

RBAC menu rendering

Navigation built from the signed-in permission set

Silent session refresh

Token auto-renew + idle-timeout warning dialog

outbound request ▼

4 Enforced at the API edge

Client-side checks are never trusted alone

Bearer interceptor

Attaches the access token to opted-in API calls

Private API gateway

Reachable only through the in-VPC endpoint

PRIVATE

Lambda authorizer

Re-verifies JWT signature / issuer / audience server-side

Grant & serve

On valid token, request reaches the backend data plane

OIDC / PKCE · claim-based RBAC · defense-in-depth authorization

One static build boots, resolves environment and identity from the subdomain, authenticates via OIDC + PKCE, then RBAC-gates the client — while the private API re-verifies every token.

Product capability map

PRODUCT CAPABILITY MAP

What the portal delivers to payers, providers & developers

A single tenant-scoped console spanning self-service identity administration, standards-based FHIR clinical data exploration, and two analytics suites — operational API-usage insight for developers and population-health demographics for the business.

Every capability is scoped to the signed-in customer tenant Tenant identity travels in the token and constrains all data, clinical and reporting queries

1 Identity & access administration

Self-service RBAC and OAuth application onboarding

User management

Create, edit and view users; assign roles

Roles & permissions

Full RBAC — roles, permission sets, assignment

OAuth app registration

Register client apps with redirect URIs & scopes

developer portal

data & insight surfaces ▼

2 FHIR clinical data explorer

Standards-based clinical record browsing, tenant-scoped

FHIR R4

clinical data

Patient search & profile

Find members and open a consolidated clinical view

15 FHIR R4 resource types

Conditions, encounters, medications, labs, procedures, care plans & more

HL7 FHIR R4

Paged, searchable reads

Code/date filters and pagination over each resource

analytics built on the same data ▼

3 Operational & population analytics

Two dashboard suites over the reporting APIs

Chart.js

dashboards

API usage analytics

Status-code, response-time & data-volume trends by API & resource

developer insight

Member demographics

Enrollment by age, gender, race, county & coverage type

population health

Interactive charts

Reusable filter + chart component library

Multi-tenant · role-based · FHIR-native · analytics-rich

A single tenant-scoped console: identity & access administration, a FHIR R4 clinical-data explorer, and two analytics suites.

Delivery & platform engineering

DELIVERY & PLATFORM ENGINEERING

Cross-cloud CI/CD over reusable infrastructure as code

Reusable nested-stack CloudFormation provisions the hosting estate and publishes service-discovery parameters; an Azure DevOps pipeline builds once and promotes through gated environments into AWS — all on top of a shared, versioned Angular design-system and authentication library.

1 Reusable infrastructure as code

Nested CloudFormation modules, provisioned once per environment

CloudFormation WAF SSM

Bootstrap

Deployment IAM identity + template bucket

Reusable modules

One parameterized template each for storage, CDN & parameters

nested stacks

Provisioned estate

Origin buckets, CDN distributions, WAF ACLs, access logs

Service-discovery params

Bucket names & distribution IDs written to SSM

SSM → CI/CD

SSM parameters hand off to the pipeline ▼

2 Cross-cloud delivery pipeline

Azure DevOps as control plane, deploying to AWS

Azure DevOps SAM S3 + CloudFront

Build once

Install & produce an optimized SPA artifact on master

build pipeline

Promote with gates

dev → qa → uat → prod, each behind manual approval

release pipeline

Deploy SPA

Sync artifact to origin bucket, then invalidate the CDN

Deploy serverless

SAM deploy of the API/Lambda stack per stage

shared foundation beneath both portals ▼

3 Shared Angular library workspace

A published design system + auth layer, packaged for reuse

ng-packagr monorepo

Design-system library

40+ Material-based UI components, dialogs, tables & theming

component library

Okta auth library

Session lifecycle, token & guard services shared across apps

auth library

Write-once, deploy-anywhere

Same build targetable at CDN, containers, Kubernetes or PaaS

laC modules · gated promotion · shared libraries · portable frontend

Reusable nested-stack infrastructure-as-code and a cross-cloud build-once / promote-through-gates pipeline, over shared Angular design-system and auth libraries.

A secure, compliant, multi-tenant portal platform — and a reusable foundation

2 portals, 1 build

Admin + developer/sandbox portals from a single runtime-configured artifact

15 FHIR R4

Clinical resource types in a tenant-scoped, standards-based explorer

Private by default

VPC-only API, private CDN origins, secrets injected at deploy time

4 gated stages

dev → QA → UAT → prod, promoted across separate cloud accounts

Defense-in-depth by construction. OIDC + PKCE authentication, a signed permission claim driving client-side RBAC, and an independent server-side Lambda authorizer over a private API gateway mean PHI is never guarded by the client alone. HITRUST-aligned audit forwarding and X-Ray tracing keep behavior traceable.

Built to scale and to reuse. A versioned design-system library and a companion authentication library are packaged and consumed across applications, and the optimized frontend build is targetable at CDN, container, Kubernetes, or PaaS — so the platform can grow from one internal console to a public developer ecosystem without rework, on a foundation reusable across products.